Roofline Analysis and Kernel Fusion of Window Attention in TinyViT

Anurag Dhiman

November 1, 2026

Abstract

Window attention is the dominant cost center in TinyViT, a vision transformer built for edge deployment, yet its low nominal FLOP count makes it easy to dismiss as already cheap. We show that on Apple M4's unified-memory GPU this intuition is wrong in a specific, measurable way: standard window attention is not compute-bound at all, and the two most natural fixes , framework-level kernel fusion (MLX, `torch.compile`) and a hand-written Metal kernel using Apple's hardware matrix-tile instruction , both fail to beat PyTorch's native implementation for the larger window configuration TinyViT actually uses. The bottleneck we eventually isolate is neither raw compute nor dispatch overhead in the abstract, but a concrete, previously-unmeasured inefficiency: a 25× redundant re-read of the key/value tensors from device memory. A kernel that caches K/V once per window in on-chip memory removes 96% of that traffic and wins end-to-end on all three TinyViT scales (5M/11M/21M parameters), verified on real pretrained weights and a 20,000-image ImageNet-1k evaluation with no measurable accuracy cost.

1 Introduction

Vision transformers are increasingly deployed on consumer and edge hardware ,phones, tablets, laptops , where inference latency and memory traffic matter as much as top-1 accuracy. TinyViT [1] targets this regime directly: it replaces global self-attention with local *window attention* (following Swin Transformer [2]), partitioning the feature map into small non-overlapping windows (here, $7 \times 7 = 49$ or $14 \times 14 = 196$ tokens) and running attention independently within each. This cuts the quadratic cost of global attention to linear in the number of windows, and because each window is small, the nominal FLOP count of the attention core is tiny relative to the convolutional stem and MLP blocks that dominate TinyViT's parameter count.

That FLOP count is also exactly why window attention is easy to get wrong: a small operation run many times is a textbook setting for memory-bandwidth and dispatch overhead to dominate instead of compute, and on Apple Silicon's unified-memory architecture neither cost is visible to a FLOP count. This project asks: *how much of TinyViT's window-attention latency on an Apple M4 is compute, memory traffic, or dispatch overhead, and can a fused kernel targeting the real bottleneck beat standard PyTorch end-to-end, on the real model, without sacrificing accuracy?*

The honest answer required five successive kernel designs before it was yes, each failure ruling out a plausible hypothesis about where the time was going. The final design's win comes from a bottleneck that has nothing to do with matrix-multiply throughput , the same insight underlying FlashAttention [5], applied here to Apple's `simdgroup_matrix` hardware primitive rather than CUDA warps. Section 3 walks through this history; Section 4 reports the final, verified result.

2 Prior Work

Vision transformers and window attention. The Transformer [4] and its vision adaptation, ViT [3], established global self-attention for images at quadratic cost in token count. Swin Transformer [2] introduced windowed, shifted local attention to make this linear; TinyViT [1] combines a Swin-style window attention stage with distillation-based pretraining to produce small, fast models (5M/11M/21M parameters) that are nonetheless competitive on ImageNet-1k. This project uses TinyViT's publicly released pretrained checkpoints (via `timm`) unmodified as its baseline and ground truth.

Performance modeling. The Roofline model [6] relates achievable throughput to arithmetic intensity (FLOPs per byte moved) via a measured peak compute rate $P_{\max}$ and peak bandwidth $B_{\max}$. We build an *empirical* roofline for the M4 GPU (matmul sweep for $P_{\max}$, elementwise-op sweep for $B_{\max}$) rather than relying on vendor specifications, since Apple does not publish the relevant per-precision numbers, and because dispatch overhead , invisible to the classical model , turns out to matter as much as either axis.

Kernel fusion frameworks and primitives. MLX [8] is Apple's array framework for Apple Silicon, offering fused primitives (e.g. `mx.fast.scaled_dot_product_attention`) as an alternative to composing PyTorch ops under the MPS backend [9]; we also compare against `torch.compile` [7], PyTorch's own fusion compiler. Where both are insufficient (Section 3.1), we drop to Apple's Metal Shading Language [10] directly, using `simdgroup_matrix` , Apple GPUs' hardware-accelerated 8×8 matrix-tile operations, analogous to NVIDIA's tensor cores. The final kernel's algorithmic structure an online/streaming softmax that never materializes the full score matrix — is a from-scratch reimplementation of FlashAttention's core idea [5], adapted to Apple's SIMD-group model rather than CUDA thread blocks.

3 Idea and Implementation

The approach follows an eight-phase empirical pipeline: profile the unmodified baseline (Phase 1); build an empirical roofline (Phase 2) and map each stage's arithmetic intensity onto it (Phase 3); microbenchmark window attention in isolation (Phase 4); implement and iterate a fused kernel (Phase 5); ablate it against compiler-level optimization (Phase 6); verify correctness and real accuracy (Phase 7); and consolidate into a final, honestly-reported benchmark (Phase 8). Phases 1–6 are covered in this section; Phases 7–8 in Section 4.

3.1 Phases 1-3: baseline profiling and empirical roofline

Phase 1 profiles the standard, unmodified TinyViT-5M forward pass under `torch.profiler` (4.97 ± 0.21 ms/iter at batch 1). Per-block self-time: layout ops (reshape/permute/window partition) 26.1%, MLP 22.1%, depthwise conv 15.3%, attention core (scores+bias+softmax+AV) 10.8%, projections 13.0%, LayerNorm 6.8%, residual-add 6.0% , layout plumbing alone outweighs the attention math it surrounds, an early hint that overhead, not FLOPs, drives this story. Phases 2–3 make it precise: rather than rely on vendor specifications, we measure the GPU's peak compute rate $P_{\max}$ and peak bandwidth $B_{\max}$ directly: a float32 matmul-size sweep gives $P_{\max} = 2576$ GFLOP/s (fp16: 2836 GFLOP/s), a large-elementwise-op sweep gives $B_{\max} = 86.6$ GB/s (fp32), giving a ridge point $I_{\text{ridge}} = P_{\max}/B_{\max} = 29.7$ FLOPs/byte (the equivalent fp16 bandwidth measured *lower* than fp32 despite moving half the bytes per element , reproducible, but attributed to a PyTorch/MPS kernel-efficiency artifact rather than a true hardware ceiling, so fp32 is used as the primary roofline). Table 1 maps TinyViT-5M's actual per-stage arithmetic intensity , isolated microbenchmarks on real captured shapes and weights from the model's first block , onto this roofline.

Stage	AI (FLOPs/B)	Achieved GFLOP/s	% of own bound	Regime
Conv stem	18.65	279	18.7%	memory-bound
Depthwise (local) conv	2.18	10	5.5%	memory-bound
MLP	36.40	929	39.0%	mixed
QKV projection	34.95	605	25.3%	mixed
Output projection	26.71	290	12.6%	memory-bound
Attention core ($QK^\top$+softmax+AV)	6.30	298	53.8%	memory-bound
Window partition / reverse	≈ 0	≈ 0	–	pure reshape

Table 1: Per-stage arithmetic intensity vs. the measured ridge point ($I_{\text{ridge}} = 29.7$), TinyViT-5M, batch size 1.

Every stage sits far below the ridge point: the whole model, not only attention, is memory-bound on this hardware at these shapes. The attention core's AI (6.30 averaged across window sizes; 7.15 at 14×14 specifically, N=196) is the lowest non-trivial value of any stage, yet it already reaches the *highest* utilization of its own bound (53.8%) among the compute stages , its ceiling is low, not left on the table within that ceiling. Since AI's FLOPs numerator is fixed by the attention computation itself, raising it means cutting the bytes-moved denominator , the one thread connecting every design in Section 3.4: designs (1)-(2) chased the numerator (faster matmul) and failed; Section 3.5's win is the one design that directly and substantially cuts the denominator instead.

3.2 Phase 4: window-attention microbenchmark

Before building any fused kernel, window attention is isolated and swept across window size, embedding dimension, and batch size (18 configurations around a baseline of window size 7, dim 96, 16 windows, batch 4), decomposing total latency into QKV projection, reshape, $QK^\top$, softmax, AV, and output projection. Table 2 shows the window-size axis, holding the rest fixed.

N	Total (μs)	Dispatch %	$QK^\top$ (μs)	Softmax (μs)	AV (μs)
16	267.6	20.3%	54.3	19.1	19.7
49	563.4	16.5%	184.3	73.9	37.5
196	2983.0	0.0%	2670.5	1573.0	467.8

Table 2: Window-attention microbenchmark, window size $N \in \{16, 49, 196\}$ (4×4, 7×7, 14×14), dim 96, 16 windows, batch 4. Dispatch % is PyTorch/MPS dispatch overhead's estimated share of total latency.

This establishes, ahead of any kernel design, that dispatch overhead's *share* of total time shrinks from 20.3% to 0.0% as N grows ($16 \to 196$) , at N=16, a fifth of the time is overhead, exactly where removing dispatch alone can win , while $QK^\top$ time grows quadratically with N. TinyViT's two window sizes are genuinely different regimes, not the same problem at different scales, motivating testing both separately throughout this section. The other two axes confirm this is N-specific: batch size (1/4/16/32, N=49) scales total time roughly linearly (198/563/2332/5053 μs) with no consistent dispatch-share trend, and embedding dimension (64-576) shows dispatch share *falling* as it grows (∼17%→8%) since compute scales with dim but per-call overhead does not , N, not batch or dim, sets the regime.

3.3 Phase 5a: why framework-level fusion is not enough

Section 3.1's memory-bound finding motivates a natural first fix: framework-level fusion, replacing the baseline's sequence of separately-dispatched PyTorch ops (QKV projection, window partition, $QK^\top$, softmax, AV, window reverse, output projection) with a single fused MLX call, or letting `torch.compile` fuse the graph automatically.

Neither works (Phase 6's first ablation). Across the project's ablation grid (window size × embedding dimension × batch size), MLX-fused attention averages **0.68**× and `torch.compile` averages **0.90**× relative to the unmodified PyTorch/MPS baseline , both net losses. Instrumenting MLX's own dispatch path isolates why: a fixed ∼300-400 μs Python-to-Metal dispatch floor that does not shrink with problem size, and that is already larger than TinyViT's own per-window compute at realistic shapes. Fusing more work into one MLX call does not amortize this floor fast enough, because the floor is paid once per call regardless of how much work that call does. This result rules out *framework-level* fusion as the fix and motivates writing a native Metal kernel directly, bypassing both PyTorch's dispatcher and MLX's Python layer.

3.4 Phase 5b: five designs, five lessons

A native kernel removes the dispatch floor, but introduces a new question: can hand-written Metal code match a vendor GEMM/attention implementation that has had years of tuning? Table 3 summarizes five successive designs, each built in direct response to the previous one's failure, for TinyViT's 14×14-window configuration ($N = 196$ tokens, head dimension $D = 32$), the configuration that *loses* under every naive approach, in contrast to the 7×7 window ($N = 49$), where a simple native kernel already wins (1.13-1.34×, Section 4) because its smaller absolute compute time keeps it in the dispatch-overhead-dominated regime where removing Python/MLX overhead is sufficient on its own.

Design (2) rules out "our matmul is slow": even *correct* hardware-matrix-unit code does not beat PyTorch's GEMM. Design (3) rules out textbook GEMM-blocking here: its reuse factor (tiles shared by at most 2 of 4 SIMD-groups) is too small to pay for its own staging cost. Design (4) only approaches parity once two confounds , a simplified shape and a missing bias term , are removed: an isolated microbenchmark omitting real computation can overstate speedup by 50 points.

3.5 Phase 5c: the winning design, caching K/V once per window

Designs (1)-(5) all share one structural property: the kernel dispatches one threadgroup per 8-row query tile, and that threadgroup independently re-reads the *entire* key and value tensors for its window from device memory. For N=196, there are $\lceil 196/8 \rceil = 25$ query tiles per window, so each window's K and V tensors are read from device memory **25 times over** , a redundancy no previous design had isolated or measured directly.

The final kernel restructures the dispatch so that *one threadgroup handles an entire window*, not one query tile: 25 SIMD-groups (800 threads) jointly stage K and V into on-chip threadgroup memory exactly once (half-precision, $196 \times 32 \times 2$ B = 12,544 B each, ≈24.5 KB combined, within the hardware's ∼32 KB budget), and each SIMD-group then runs the flash-attention recurrence , a running max m_i, normalizer l_i, and weighted output o_i for its own 8 query rows, updated over each 8-key block read from the *shared, cached* copy rather than device memory again. Running

Design	Idea / lesson	Result
1. Naive scalar kernel	Hand-written loops, no hardware matrix unit.	0.68–0.85×
2. `simdgroup_matrix`, 1 tile/SIMD-group	Correct use of Apple's hardware 8×8 tile instruction. Matmul throughput alone is not the lever: already near PyTorch's own GEMM.	0.80× (thin-K) / 0.94× (big-K)
3. Shared-memory-blocked GEMM	Textbook next step: cache operand tiles in on-chip memory across 4 cooperating SIMD-groups. Regressed: the scalar staging loop cost more than the ∼2× redundancy it removed at this problem size.	0.20–0.21×
4. fp32 flash attention	Online/streaming softmax so the $N \times N$ score matrix never materializes in device memory [5]. An isolated microbenchmark win of 1.49× over PyTorch's fused SDPA collapsed once the real learned position bias and N=196 boundary masking were restored.	0.77-0.97×
5. fp16 flash attention	Same per-tile dispatch as (4), half-precision tiles.	∼1.0× at one batch size only

Table 3: Design history for the 14×14-window attention core. Each row is gated on correctness (max-abs-difference against the unmodified baseline) before any speed number is trusted.

statistics stay in per-thread registers to leave headroom for the cache; a small scratch buffer relays `simdgroup_store` results before they fold into the accumulator. Matrix tiles use half-precision `simdgroup_half8x8`; softmax statistics stay float32 for stability. N=196 is not a multiple of 8, so the final query tile (4 rows) and key block (4 keys) are masked explicitly rather than read out of bounds.

This change is purely a data-placement change , it computes bit-identical results to design (5) at the isolated-kernel level (0.0 difference) , yet it is the only one of six designs that beats the baseline reliably, because it is the only one that directly targets the denominator identified in Section 3.1: cutting K/V bytes moved by 25× raises the attention core's effective arithmetic intensity without changing a single FLOP. Section 4 reports the end-to-end consequence of that shift, not matmul throughput, GEMM blocking, or dispatch overhead in the abstract.

4 Evaluation

Phases 7-8: correctness and real-accuracy verification, then the consolidated speedup/throughput report. All results in this section use the real pretrained TinyViT checkpoints (5M/11M/21M parameters, `timm`'s `.dist_in22k_ft_in1k` weights) with the final kernel (Section 3.5) patched into the 14×14-window attention blocks; the 7×7 blocks already use a simple native-dispatch fusion that wins on its own (1.13-1.34×, below). Every speed number is gated on a correctness check first, per the project's standing methodology.

Correctness. Table 4 reports logit-level max-absolute-difference and top-1 agreement against each model's unmodified baseline, on real inputs. All three scales pass at fp16 noise level (10^{-4}–10^{-3}), rising slightly with model size but far from a concern.

Model	Max abs. diff. (logits)	Top-1 match
TinyViT-5M	2.80×10^{-4}	True
TinyViT-11M	7.80×10^{-4}	True
TinyViT-21M	1.07×10^{-3}	True

Table 4: Correctness vs. unmodified baseline, real pretrained weights.

End-to-end speed. Table 5 reports full-model speedup across batch sizes 1-64, over 12-18 repeated trials per cell. Two pitfalls, both specific to many trials back-to-back on a fanless laptop GPU, had to be fixed first: cross-configuration thermal carryover (subprocess isolation per model) and *within*-run thermal drift at one batch size (a monotonic decline across trials, fixed with a per-trial cooldown long enough for that model's compute - 3 s for 5M, 6 s for the heavier 11M/21M). Corrected, every cell shows a confident win.

Real-world accuracy. Table 6 reports top-1/top-5 ImageNet-1k accuracy on a stratified 20,000-image subset (20/class, streamed, never downloaded to disk), fused predictions vs. each model's own baseline on the same images. Every delta is within ±0.01 points , the same noise-level magnitude this project established for plain fp16 elsewhere , with 99.98% exact top-1 agreement on all three models: the fused kernel costs no real accuracy.

Model	bs=1	bs=4	bs=16	bs=32	bs=64
TinyViT-5M	1.019×	1.032×	1.035×	1.055×	1.047×
TinyViT-11M	1.076×	1.049×	1.027×	1.062×	1.049×
TinyViT-21M	1.026×	1.042×	1.025×	1.032×	1.028×

Table 5: End-to-end speedup vs. unmodified PyTorch/MPS baseline (higher is better; n=12–18 trials per cell, mean reported, all $\sigma \leq 0.064$).

Model	Baseline top-1/top-5	Fused top-1/top-5	Δtop-1 (pp)	Agreement
TinyViT-5M	80.905% / 95.645%	80.895% / 95.650%	−0.010	99.98%
TinyViT-11M	83.035% / 96.535%	83.040% / 96.530%	+0.005	99.98%
TinyViT-21M	85.035% / 97.250%	85.040% / 97.250%	+0.005	99.98%

Table 6: Real ImageNet-1k accuracy, 20,000 images, fused kernel vs. unmodified baseline.

Absolute latency and throughput. Table 7 grounds Table 5's ratios in absolute units: real wall-clock compute time over the full 20,000-image run underlying Table 6,

Model	Baseline (ms/img)	Fused (ms/img)	Baseline (img/s)	Fused (img/s)
TinyViT-5M	7.548	6.032	132.5	165.8
TinyViT-11M	8.876	8.168	112.7	122.4
TinyViT-21M	14.237	13.413	70.2	74.6

Table 7: Sustained throughput, full 20,000-image ImageNet-1k run (real compute time.

Memory traffic and kernel launches, quantified. N=196 and D=32 are identical across all three scales (only num_heads differs: 5/8/12), so the reduction is architecture-determined: the old per-tile design moves 627,200 B of K/V traffic per (window, head); the cached design moves 25,088 B , 25× (96.0%), or 2.87/4.59/6.89 MiB per batch-size-1 forward pass on 5M/11M/21M , the concrete fix for Section 3.1's low-AI diagnosis. Separately, `torch.profiler` counts 7 dispatcher-visible ops for the baseline attention core (split, expand, transpose, reshape, clone, SDPA, its MPS fallback); the new kernel replaces all seven with one `dispatchThreadgroups` call , 7× fewer launches.

Comparison with other baselines. Two other optimization paths evaluated end-to-end are weaker than the unmodified baseline comparison above: (i) the best pre-existing PyTorch-level combination (conv+batch-norm fusion + channels-last + fp16) reaches 1.49× in isolation but never touches the attention core, (ii) fusing only the 7×7-window core (a per-block win, 1.13-1.34×) covers just ≈9.7% of model time, so its end-to-end effect (0.96-1.01×) is noise. Tables 5–6's design is the only one here that survives averaging into whole-model latency.

Conclusion

TinyViT's window attention on Apple M4 is not slow because of its FLOP count: framework-level fusion and a native kernel around the GPU's hardware matrix-tile instruction both fail for the 14×14-window configuration that matters most. The gap is a specific, measured inefficiency (25× redundant key/value re-reads), and a kernel removing exactly that redundancy wins reliably, end-to-end, on real pretrained weights, across three model scales, with no measurable accuracy cost on 20,000 real ImageNet-1k images.

References

[1] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu, and L. Yuan. TinyViT: Fast Pretraining Distillation for Small Vision Transformers. In *European Conference on Computer Vision (ECCV)*, 2022.

[2] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[6] S. Williams, A. Waterman, and D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[7] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[8] Apple Machine Learning Research. MLX: An array framework for Apple silicon. https://github.com/ml-explore/mlx, 2023.

[9] Apple Inc. Metal Performance Shaders / PyTorch MPS Backend Documentation. https://developer.apple.com/metal/pytorch/.

[10] Apple Inc. Metal Shading Language Specification. https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf.

[11] A. Dhiman. Roofline Analysis and Kernel Fusion of TinyViT Window Attention — source code, scripts, and full results. PLACEHOLDER-INSERT-REPOSITORY-URL-HERE, 2026.